

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

PRIYANSHU SHARAN (1WA24CS223)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by PRIYANSHU SHARAN (1WA24CS223), who is a Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl No	Experiment Title	Page No
1	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	5
2	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	39
3	Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one) a) Rate-Monotonic b) Earliest-Deadline First c) Proportional Scheduling	46
4	Write a C program to simulate: (Any one) a) Producer-Consumer problem using semaphores b) Dining-Philosopher's problem	62
5	Write a C program to simulate: (Any one) a) Bankers' algorithm for the purpose of deadlock avoidance b) Deadlock Detection	79
6	Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-Fit b) Best-Fit c) First-Fit	100
7	Write a C program to simulate page replacement algorithms. (Any one) a) FIFO b) LRU c) Optimal	112

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.



Name PRYANSHU SHARAN Std _____ Sec _____
 Roll No. 1WA24C3228 Subject _____ School/College BMSCE
 School/College Tel. No. _____ Parents Tel. No. 9142533317

Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
1	27/2	PRACTICE PROGRAM	1	-10
2	6/3	LAB PROGRAM 1-(FCFS)	8	-10
	6/3	- SJF Non Preemptive	5	-10
	6/3	- SJF Preemptive	7	-10
	13/3	- Priority Non Preemptive	9	-10
	13/3	- Priority Preemptive	12	-10
	13/3	- Round Robin	15	-10
3	07/3	LAB PROGRAM 2 - MULTI LEVEL QUEUE SCHEDULING ALGORITHM		27/3
4	27/3	LAB PROGRAM - 3 RATE MONOTONIC SCHEDULING ALGORITHM		
5	27/3	EARLIEST DEADLINE FIRST		
6	27/3	PROPORTIONAL SHARE SCHEDULING		
		- DOLING	-10	
7	10/4	LAB PROGRAM - 4 PRODUCER CONSUMER PROBLEM	-10	
8	24/4	DINING PHILOSOPHER PROBLEM	-10	
9	8/5	BANKER'S ALGORITHM		16 (10)
10	8/5	DEADLOCK ALGORITHM		
11	14/5	FIRST BEST WORST FIT		-10
15	22/5	FIFO OPTIMAL LRU		-10

Program - 1

Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

Code:

=>FCFS:

```
#include <stdio.h>

int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n];

    printf("Enter Arrival Times:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Times:\n");
```

```

for(i = 0; i < n; i++)
    scanf("%d", &bt[i]);

ct[0] = at[0] + bt[0];

for(i = 1; i < n; i++)
{
    if(ct[i-1] < at[i])
        ct[i] = at[i] + bt[i];
    else
        ct[i] = ct[i-1] + bt[i];
}

float avg_tat = 0, avg_wt = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    avg_tat += tat[i];
    avg_wt += wt[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

```

```

printf("\nAverage Turnaround Time = %.2f", avg_tat / n);
printf("\nAverage Waiting Time = %.2f\n", avg_wt / n);

return 0;
}

```

Result:

USN : 1WA24CS223
Name: Priyanshu Sharan

Enter number of processes: 5
Enter Arrival Times:
0 2 4 6 8
Enter Burst Times:
3 6 4 5 2

PID	AT	BT	CT	TAT	WT
P1	0	3	3	3	0
P2	2	6	9	7	1
P3	4	4	13	9	5
P4	6	5	18	12	7
P5	8	2	20	12	10

Average Turnaround Time = 8.60
Average Waiting Time = 4.60
priyanshusharan@Priyanshus-MacBook-Air os-coding %

date 5/3/24
Page 13

LAB PROGRAM 1

- ① Write the C program to simulate the following CPU Scheduling algorithm to find turnaround time and waiting time

- a) FCFS b) Priority
c) SJF d) Round Robin

a) FCFS

```
#include <stdio.h>
int main() {
    int n, i;
    int at[20], bt[20], ct[20], wt[20], tt[20];
    printf("Enter the number of processes:");
    scanf("%d", &n);
    printf("Enter Arrival Time and Burst Time:");
    for(i=0; i<n; i++) {
        printf("P%d AT: ", i+1);
        scanf("%d", &at[i]);
        printf("P%d BT: ", i+1);
        scanf("%d", &bt[i]);
    }
    ct[0] = at[0] + bt[0];
    for(i=1; i<n; i++) {
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }
```



```

}
for (i=0; i<n; i++)
    tat[i] = c + (i-1)*t;
for (i=0; i<n; i++)
    wt[i] = tat[i] - t;

float avgwt = 0, avgtat = 0;
for (i=0; i<n; i++) {
    avgwt += wt[i];
    avgtat += tat[i];
}

printf("Avg tat: %.2f", avgtat/n);
printf("Avg wt: %.2f", avgwt/n);
return 0;
}

```

Output

Enter the number of processes: 4
Enter Arrival Time and Burst Time

P1 AT: 1
P1 BT: 3
P2 AT: 4
P2 BT: 8
P3 AT: 2
P3 BT: 3
P4 AT: 5
P4 BT: 4

Avg tat = 9.50
Avg wt = 5.00

P1	P3	P2	P4
7	15	19	

=> SJF Non Preemptive

```
#include <stdio.h>

int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int n, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], wt[n], tat[n], p[n];

    printf("Enter Burst Times:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        p[i] = i + 1;
    }

    // Sort processes according to Burst Time
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(bt[i] > bt[j])
            {
                int temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
}
```

```

        temp      =
        p[i];    p[i]
        =        p[j];
        p[j]      =
        temp;
    }
}

}

wt[0] = 0;

for(i = 1; i < n; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
}
float avg_wt = 0, avg_tat = 0;
printf("\nPID\tBT\tWT\tTAT\n");
for(i = 0; i < n; i++)
{
    tat[i] = bt[i] + wt[i];

    avg_wt += wt[i];
    avg_tat += tat[i];

    printf("P%d\t%d\t%d\t%d\n",
           p[i], bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

RESULT

USN : 1WA24CS223

Name: Priyanshu Sharan

Enter number of processes: 5

Enter Burst Times:

6 2 8 3 4

PID	BT	WT	TAT
P2	2	0	2
P4	3	2	5
P5	4	5	9
P1	6	9	15
P3	8	15	23

Average Waiting Time = 6.20

Average Turnaround Time = 10.80

priyanshusharan@Priyanshus-MacBook-Air os-coding %

⑥ SJF Non Preemptive

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i, j;
```

```
    printf ("Enter number of processes");
```

```
    scanf ("%d", &n);
```

```
    int at[n], bt[n], ct[n], tat[n], wt[n]
```

```
    int finished[n]
```

```
    for (i=0; i<n; i++) {
```

```
        printf ("Enter Arrival Time and Burst  
        Time for P %d: ", i+1);
```

```
        scanf ("%d %d", &at[i], &bt[i]);
```

```
        finished[i] = 0;
```

```
    }
```

```
    int current_time = 0;
```

```
    int computed = 0;
```

```
    while (computed < n) {
```

```
        int min_bt = 9999;
```

```
        int index = -1;
```

```
        for (i=0; i<n; i++) {
```

```
            if (at[i] <= current_time
```

```
                & finished[i] == 0) {
```

```
                if (bt[i] < min_bt) {
```

```
                    min_bt = bt[i];
```

```
                    index = i;
```

```
            }
```

```
        }
```

```
        if (index == -1) {
```

```
            if (index == -1) {
```

```
                current_time++;
```

```
            }
```


else {

ct[index] = current_time + bt[index];

tat[index] = ct[index] - at[index];

wt[index] = tat[index] - bt[index];

current_time = ct[index];

finished[index] = 1;

completed++;

}

printf("Process\tAT\tBT\tCT\tTAT\tWT\n");

for (i = 0; i < n; i++) {

printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i],

bt[i], ct[i], tat[i], wt[i]);

}

}

return 0;

}

Output

P1 P2 P3 P4

Enter the no of process : 4

Enter Arrival Time and Burst Time for P1: 0 7

Enter Arrival Time and Burst Time for P2: 8 3

Enter Arrival Time and Burst Time for P3: 3 4

Enter Arrival Time and Burst Time for P4: 5 6

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	8	3	14	6	3
P3	3	4	11	8	4
P4	5	6	20	15	9

=> SJF Preemptive

```
#include <stdio.h>
#include <limits.h>

int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rt[n];
    int ct[n], tat[n], wt[n];

    printf("Enter Arrival Times:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Times:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d",
            &bt[i]); rt[i] =
            bt[i];
    }

    int completed = 0;
    int currentTime = 0;

    while(completed < n)
    {
        int shortest = -1;
        int minRT = INT_MAX
```

```

for(int i = 0; i < n; i++)
{
    if(at[i] <= currentTime &&
        rt[i] > 0 &&
        rt[i] < minRT)
    {
        minRT = rt[i];
        shortest = i;
    }
}

if(shortest == -1)
{
    currentTime++;
    continue;
}

rt[shortest]--;
currentTime++;

if(rt[shortest] == 0)
{
    completed++;

    ct[shortest] = currentTime;
    tat[shortest] = ct[shortest] - at[shortest];
    wt[shortest] = tat[shortest] - bt[shortest];
}
}

float avgWT = 0, avgTAT = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

```



```

for(int i = 0; i < n; i++)
{
    avgWT += wt[i];
    avgTAT += tat[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n"
           , i + 1,
           at[i],
           bt[i],
           ct[i],
           tat[i],
           wt[i]);
}

printf("\nAverage Waiting Time = %.2f", avgWT / n);
printf("\nAverage Turnaround Time = %.2f\n", avgTAT / n);

return 0;
}

```

Result

USN : 1WA24CS223
Name: Priyanshu Sharan

Enter number of processes: 4
Enter Arrival Times:
0 1 2 3
Enter Burst Times:
8 4 2 1

PID	AT	BT	CT	TAT	WT
P1	0	8	15	15	7
P2	1	4	8	7	3
P3	2	2	4	2	0
P4	3	1	5	2	1

Average Waiting Time = 2.75
Average Turnaround Time = 6.50

priyanshusharan@Priyanshus-MacBook-Air os-coding %

③ SJF Preemptive

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i, j;
```

```
    printf("Enter the number of process: ");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], rt[n], ct[n], tat[n], wq[n];
```

```
    for(i=0; i<n; i++) {
```

```
        printf("Enter Arrival Time and Burst Time
```

```
        for P-%d: ", i+1);
```

```
        scanf("%d %d", &at[i], &bt[i]);
```

```
        rt[i] = bt[i];
```

```
    }
```

```
    int current_time = 0, completed = 0;
```

```
    while (completed < n) {
```

```
        int min_rt = 9999, index = -1;
```

```
        for (i=0; i<n; i++) {
```

```
            if (at[i] <= current_time && rt[i] > 0)
```

```
                if (rt[i] < min_rt) {
```

```
                    min_rt = rt[i];
```

```
                    index = i;
```

```
                }
```

```
            }
```

```
        }
```

```
        if (index == -1) {
```

```
            current_time++;
```

```
        }
```

```
        else {
```



```

rt[index] = 0;
current_time++;
}
if (rt[index] == 0) {
    completed++;
    ct[index] = current_time;
    tat[index] = ct[index] - at[index];
    wt[index] = tat[index] - bt[index];
}
}
printf("In P\t AT\t BT\t CT\t TAT\t WT\n");
for (i = 0; i < n; i++) {
    printf("P%d\t %d\t %d\t %d\t %d\t %d\n", i+1, at[i],
        bt[i], ct[i], tat[i], wt[i]);
}
return 0;
}

```

Output

P1	P2	P4	P3	P5
0	1	5	10	17

Enter number of processes: 4

Enter Arrival Time and Burst Time for P1: 0 8

Enter Arrival Time and Burst Time for P2: 1 4

Enter Arrival Time and Burst Time for P3: 2 9

Enter Arrival Time and Burst Time for P4: 3 5

P	AT	BT	CT	TAT	WT
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

Program - 1

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

→ Priority (pre-emptive & Non-pre-emptive)

→ Round Robin (Experiment with different quantum sizes for RR algorithm)

CODE

=> Priority Scheduling Non-Preemptive

```
#include <stdio.h>
int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int n, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[n], pr[n], wt[n], tat[n], p[n];

    printf("Enter Burst Times:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        p[i] = i + 1;
    }

    printf("Enter Priorities (smaller number = higher priority):\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pr[i]);

    for(i = 0; i < n - 1; i++){
        for(j = i + 1; j < n; j++){
            if(pr[i] > pr[j])
            {
```

```

        int temp;

        temp = pr[i];
        pr[i] = pr[j];
        pr[j] = temp;

        temp = bt[i];
        bt[i] = bt[j];
        bt[j] = temp;

        temp      =
        p[i];      p[i]
        =          p[j];
        p[j]      =
        temp;
    }
}

wt[0] = 0;

for(i = 1; i < n; i++)
    wt[i] = wt[i-1] + bt[i-1];

float avgWT = 0, avgTAT = 0;

printf("\nPID\tBT\tPR\tWT\tTAT\n");

for(i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];

    avgWT += wt[i];
    avgTAT += tat[i];

    printf("P%d\t%d\t%d\t%d\t%d\n",
           p[i], bt[i], pr[i], wt[i], tat[i]);
}
printf("\nAverage Waiting Time = %.2f", avgWT/n);
printf("\nAverage Turnaround Time = %.2f\n", avgTAT/n);
return 0;
}

```

Result

USN : 1WA24CS223

Name: Priyanshu Sharan

Enter number of processes: 5

Enter Burst Times:

10 5 8 6 2

Enter Priorities (smaller number = higher priority):

3 1 4 2 5

PID	BT	PR	WT	TAT
P2	5	1	0	5
P4	6	2	5	11
P1	10	3	11	21
P3	8	4	21	29
P5	2	5	29	31

Average Waiting Time = 13.20

Average Turnaround Time = 19.40

priyanshusharan@Priyanshus-MacBook-Air os-coding %

④ Priority Non-Preemptive

#include <stdio.h>

(NOTE

Number = 1 Parameter

int main() {

int n, i;

int pid[20], at[20], bt[20], pr[20];

int wt[20], tat[20], ct[20];

int completed[20] = {0};

int current_time = 0, completed_count = 0;

int highest_priority, selected;

float avg_wt = 0, avg_tat = 0;

printf("W A 24 CS 223\n");

printf("Enter number of processes: ");

scanf("%d", &n);

for (i = 0; i < n; i++) {

pid[i] = i + 1;

printf("P %d Arrival Time: ", i + 1);

scanf("%d", &at[i]);

printf("P %d Burst Time: ", i + 1);

scanf("%d", &bt[i]);

printf("P %d Priority: ", i + 1);

scanf("%d", &pr[i]);

while (completed_count < n) {

highest_priority = 999;

selected = -1;

for (i = 0; i < n; i++) {

if (at[i] <= current_time &&

completed[i] == 0) {

if (pr[i] < highest_priority) {

highest_priority = pr[i];

selected = i;


```

    }
}

if (selected == 1) {
    current_time++;
}
else {
    ct[selected] = current_time;
    tat[selected] = ct[selected] - at[selected];
    wt[selected] = tat[selected] - bt[selected];
    current_time = ct[selected];
    completed_count++;
    avg_wt += wt[selected];
    avg_tat += tat[selected];
}

}

printf("In Process | P ID | BT | PR | CT | TAT\n");

for (i = 0; i < n; i++) {
    printf("P %d | %d | %d | %d | %d | %d | %d | %d | %d | %d\n",
        pid[i], at[i],
        bt[i], pr[i], ct[i], tat[i], wt[i]);
}

avg_wt /= n;
avg_tat /= n;
printf("In Average Waiting Time = %.2f",
    avg_wt);
printf("In Average Turnaround Time %.2f\n",
    avg_tat);

return 0;
}

```


Output

Enter number of Processes: 5

P1 Arrival Time: 0

P1 Burst Time: 3

P1 Priority Time: 5

P2 Arrival Time: 2

P2 Burst Time: 2

P2 Priority Time: 3

P3 Arrival Time: 3

P3 Burst Time: 5

P3 Priority Time: 2

P4 Arrival Time: 4

P4 Burst Time: 4

P4 Priority Time: 4

P5 Arrival Time: 6

P5 Burst Time: 1

P5 Priority Time: 1

Process	AT	BT	PR	CT	TAT	WT
P1	0	3	5	3	3	0
P2	2	2	3	11	9	7
P3	3	5	2	8	5	0
P4	4	4	4	15	11	7
P5	6	1	1	9	3	2

Average WT: 2.20

Average TAT: 6.20

P1	P3	P5	P2	P4
0	3	8	9	15

=> Priority Scheduling Preemptive

```
#include <stdio.h>
#include <limits.h>

int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], rt[n], pr[n];
    int wt[n], tat[n], ct[n];

    printf("Enter Burst Times:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Priorities (smaller number = higher priority):\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &pr[i]);

    int completed = 0;
    int time = 0;

    while(completed < n)
    {
        int highest = -1;
```

```

int minPriority = INT_MAX;

for(int i = 0; i < n; i++)
{
    if(rt[i] > 0 && pr[i] < minPriority)
    {
        minPriority = pr[i];
        highest = i;
    }
}

rt[highest]--;
time++;

if(rt[highest] == 0)
{
    completed++;
    ct[highest] = time;
}
}
float avgWT = 0, avgTAT = 0;
printf("\nPID\tBT\tPR\tCT\tTAT\tWT\n");
for(int i = 0; i < n; i++)
{
    tat[i] = ct[i];
    wt[i] = tat[i] - bt[i];

    avgWT += wt[i];
    avgTAT += tat[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
           i+1, bt[i], pr[i], ct[i], tat[i], wt[i]);
}

```



```

    }

    printf("\nAverage Waiting Time = %.2f", avgWT/n);
    printf("\nAverage Turnaround Time = %.2f\n", avgTAT/n);

    return 0;
}

```

Result

```

USN : 1WÁ24CS223
Name: Priyanshu Sharan

```

```

Enter number of processes: 4
Enter Burst Times:
7 4 6 2
Enter Priorities (smaller number = higher priority):
3 1 2 4

```

PID	BT	PR	CT	TAT	WT
P1	7	3	17	17	10
P2	4	1	4	4	0
P3	6	2	10	10	4
P4	2	4	19	19	17

```

Average Waiting Time = 7.75
Average Turnaround Time = 12.50

```

```

> priyanshusharan@Priyanshus-MacBook-Air os-coding % 

```

© Priority Preemption

```
#include <stdio.h>
int main()
{
    int n, i, time = 0, completed = 0;
    int pid[20], at[20], bt[20], pr[20],
        rt[20], et[20], tat[20], ct[20];
    int highest, pos;
    float avg_wt = 0, avg_bt = 0;
    printf("Enter number of process: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        pid[i] = i + 1;
        printf("P%d AT: ", i + 1);
        scanf("%d", &at[i]);
        printf("P%d BT: ", i + 1);
        scanf("%d", &bt[i]);
        printf("P%d Priority: ", i + 1);
        scanf("%d", &pr[i]);
        rt[i] = bt[i];
    }
    while (completed < n)
    {
        int highest = 999;
        int pos = -1;
        for (i = 0; i < n; i++)
        {
            if (at[i] <= time && ct[i] == 0)
            {
                if (pr[i] < highest)
                {
                    highest = pr[i];
                    pos = i;
                }
            }
        }
        time++;
        ct[pos] = 1;
        rt[pos] = bt[pos];
        et[pos] = time;
        tat[pos] = et[pos] - at[pos];
        ct[pos] = 0;
        completed++;
        avg_wt += tat[pos];
        avg_bt += rt[pos];
    }
    printf("Average waiting time: %.2f\n", avg_wt / n);
    printf("Average turn around time: %.2f\n", avg_bt / n);
}
```



```
if (pos == -1) ?
```

```
time = time + 1;
```

```
}
```

```
else ?
```

```
wt[pos] = time - at[pos];
```

```
time = time + bt[pos];
```

```
ct[pos] = time;
```

```
tat[pos] = ct[pos] - at[pos];
```

```
wt[pos] = tat[pos] - bt[pos];
```

```
avg_wt = avg_wt + wt[pos];
```

```
avg_tat = avg_tat + tat[pos];
```

```
ct[pos] = 1;
```

```
completed++
```

```
}
```

```
}
```

```
printf("In Process AT BT PR CT TAT wt\n");
```

```
for (i=0; i<n; i++) {
```

```
printf("%d\t%d\t%d\t%d\t%d\t",
```

```
pid[i], at[i],
```

```
bt[i], pr[i], ct[i], tat[i],
```

```
wt[i]);
```

```
}
```

```
avg_wt = avg_wt / n;
```

```
avg_tat = avg_tat / n;
```

```
printf("In Avg wt = %.f", avg_wt);
```

```
printf("In Avg TAT = %.f", avg_tat);
```

```
return 0;
```

```
}
```



```
if (pos == -1) ?
```

```
time = time + 1;
```

```
}
```

```
else ?
```

```
wt[pos] = time - at[pos];
```

```
time = time + bt[pos];
```

```
ct[pos] = time;
```

```
tat[pos] = ct[pos] - at[pos];
```

```
wt[pos] = tat[pos] - bt[pos];
```

```
avg_wt = avg_wt + wt[pos];
```

```
avg_tat = avg_tat + tat[pos];
```

```
ct[pos] = 1;
```

```
completed++
```

```
}
```

```
}
```

```
printf("In Process AT BT PR CT TAT wt\n");
```

```
for (i=0; i<n; i++) {
```

```
printf("%d\t%d\t%d\t%d\t%d\t",
```

```
pid[i], at[i],
```

```
bt[i], pr[i], ct[i], tat[i],
```

```
wt[i]);
```

```
avg_wt = avg_wt / n;
```

```
avg_tat = avg_tat / n;
```

```
printf("In Avg wt = %.f", avg_wt);
```

```
printf("In Avg TAT = %.f", avg_tat);
```

```
return 0;
```

```
}
```


OUTPUT

Enter number of process: 5

P1 Arrival Time: 0
P1 Burst Time: 3
P1 Priority Time: 3
P2 Arrival Time: 1
P2 Burst Time: 4
P2 Priority Time: 2
P3 Arrival Time: 2
P3 Burst Time: 6
P3 Priority Time: 4
P4 Arrival Time: 3
P4 Burst Time: 4
P4 Priority Time: 6
P5 Arrival Time: 4
P5 Burst Time: 2
P5 Priority Time: 10

Process	AT	BT	PR	CT	TAT	WT
P1	0	3	3	7	7	4
P2	1	4	2	5	4	0
P3	2	6	4	13	11	5
P4	3	4	6	17	14	10
P5	4	2	10	19	15	13

Average Waiting Time = 6.40
Average Turnaround Time = 10.20

=> Round Robin Scheduling

```
#include <stdio.h>

int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int n, tq;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], rt[n], wt[n], tat[n];

    printf("Enter Burst Times:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    int time = 0;
    int completed = 0;

    while(completed < n)
    {
        int done = 1;

        for(int i = 0; i < n; i++)
        {
            if(rt[i] > 0)
```

```

        {
            done = 0;

            if(rt[i] > tq)
            {
                time += tq;
                rt[i] -= tq;
            }
            else
            {
                time += rt[i];
                wt[i] = time - bt[i];
                rt[i] = 0;
                completed++;
            }
        }
    }

    if(done)
        break;
}
float avgWT = 0, avgTAT = 0;
printf("\nPID\tBT\tWT\tTAT\n");
for(int i = 0; i < n; i++)
{
    tat[i] = bt[i] + wt[i];

    avgWT += wt[i];
    avgTAT += tat[i];

    printf("P%d\t%d\t%d\t%d\n",
        i + 1,

```



```

        bt[i],
        wt[i],
        tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avgWT/n);
    printf("\nAverage Turnaround Time = %.2f\n", avgTAT/n);

    return 0;
}

```

Result

USN : 1WA24CS223
Name: Priyanshu Sharan

Enter number of processes: 4
Enter Burst Times:
10 5 8 6
Enter Time Quantum: 3

PID	BT	WT	TAT
P1	10	19	29
P2	5	12	17
P3	8	20	28
P4	6	17	23

Average Waiting Time = 17.00
Average Turnaround Time = 24.25

priyanshusharan@Priyanshus-MacBook-Air os-coding % 

④ Round Robin

```
#include <stdio.h>
int main() {
    int n;
    int quantum, time = 0, completed = 0;
    int queue[100], front = 0, rear = 0;
    int visited[100] = {0};
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int at[n], bt[n], rt[n], m[n], tat[n], wt[n];
    for (int i = 0; i < n; i++) {
        printf("Enter AT and BT for process %d: ", i);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }
    printf("Enter time quantum: ");
    scanf("%d", &quantum);
    for (int i = 0; i < n; i++) {
        if (at[i] <= time) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }
    while (front < rear) {
        int i = queue[front++];
        int execute = (rt[i] > quantum) ?
            quantum : rt[i];
        rt[i] -= execute;
        time += execute;
    }
}
```



```

for (int j=0; j<n; j++) {
    if (at[j] <= time && visited[j] == 0) {
        queue[rear++] = j;
        visited[j] = 1;
    }
}

if (rc[i] > 0) {
    queue[rear++] = i;
}
else {
    // if rc[i] == 0, then process i is
    // completed.
    time = time + rc[i];
    rc[i] = 0;
    return 0;
}

```

Output

Enter number of process: 5
 Enter AT and BT for process 0: 0 5
 Enter AT and BT for process 1: 1 3
 Enter AT and BT for process 2: 2 1
 Enter AT and BT for process 3: 3 2
 Enter AT and BT for process 4: 4 3
 Enter Time quantum: 2

Process	AT	BT	CT	TAT	WT
P1	0	5	3	13	8
P2	1	3	12	11	8
P3	2	1	5	8	2
P4	3	2	9	6	4
P5	4	3	14	10	7

Avg TAT = 8.6

Avg WT = 5.8

13

1. "as many as possible" time
 2. "as fast as possible" time
 3. "as cheap as possible" time
 4. "as safe as possible" time
 5. "as reliable as possible" time
 6. "as flexible as possible" time
 7. "as easy to use as possible" time
 8. "as secure as possible" time
 9. "as efficient as possible" time
 10. "as effective as possible" time
 11. "as accurate as possible" time
 12. "as precise as possible" time
 13. "as consistent as possible" time
 14. "as complete as possible" time
 15. "as thorough as possible" time
 16. "as detailed as possible" time
 17. "as in-depth as possible" time
 18. "as comprehensive as possible" time
 19. "as exhaustive as possible" time
 20. "as exhaustive as possible" time

Program - 2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE

=> Multi-Level Queue Scheduling

```
#include <stdio.h>
int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");
    int ns, nu;
    printf("Enter number of System Processes: ");
    scanf("%d", &ns);
    int sbt[20];
    printf("Enter Burst Times of System Processes:\n");
    for(int i = 0; i < ns; i++)
        scanf("%d", &sbt[i]);
    printf("Enter number of User Processes: ");
    scanf("%d", &nu);
    int ubt[20];
    printf("Enter Burst Times of User Processes:\n");
    for(int i = 0; i < nu; i++)
        scanf("%d", &ubt[i]);
    int time = 0;
    printf("\nSystem Processes (Higher Priority)\n");
    printf("PID\tBT\tCT\tTAT\tWT\n");
    for(int i = 0; i < ns; i++){
        int ct = time + sbt[i];
        int tat = ct;
        int wt = tat - sbt[i];

        printf("S%d\t%d\t%d\t%d\t%d\n",
            i + 1, sbt[i], ct, tat, wt);
    }
}
```

```

        time = ct;
    }
    printf("\nUser Processes\n");
    printf("PID\tBT\tCT\tTAT\tWT\n");

    for(int i = 0; i < nu; i++){
        int ct = time + ubt[i];
        int tat = ct;
        int wt = tat - ubt[i];

        printf("U%d\t%d\t%d\t%d\t%d\n",
               i + 1, ubt[i], ct, tat, wt);
        time = ct;
    }
    return 0;
}

```

Result

`cat /dev/queue da /usr/local/lib/python3.9/site-packages/da`

USN : 1WA24CS223

Name: Priyanshu Sharan

Enter number of System Processes: 3

Enter Burst Times of System Processes:

4 6 3

Enter number of User Processes: 2

Enter Burst Times of User Processes:

5 2

System Processes (Higher Priority)

PID	BT	CT	TAT	WT
S1	4	4	4	0
S2	6	10	10	4
S3	3	13	13	10

User Processes

PID	BT	CT	TAT	WT
U1	5	18	18	13
U2	2	20	20	18

priyanshusharan@Priyanshus-MacBook-Air os-coding %

- ① Write a C-program to stimulate multilevel queue scheduling algorithm considering following scenario. All process are divided into 1) user 2) System. System process priority is higher. Use FCFS for the queue.

C-CODE

```
#include <stdio.h>
#include <string.h>
#define MAX100

struct process {
    char pid [5];
    int arrival, burst, remaining;
    int completion, turnaround, waiting;
    int qno;
    int done;
}
```

```
int main() {
    struct process p[MAX];
    int n, i, completed = 0;
    int current_time = 0, tq;
    float total_tat = 0, total_wt = 0;
    printf("Enter number of processes")
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("p[%d].pid, "p%d", i+1);
        printf("\nProcess %s\n", p[i].pid);
        printf("Enter Arrival Time: ");
    }
```



```

9  scanf("%d", &p[i].arrival);
    printf("Enter Burst Time:");
    scanf("%d", &p[i].burst);
    p[i].remaining = p[i].burst;
    p[i].done = 0;
    printf("Enter Queue (1-System, 2-Queue):");
    scanf("%d", &p[i].qno);

```

```

printf("\nEnter time quantum for system queue");
scanf("%d", &tq);

```

```

while (completed < n) {

```

```

    count_executed = 0;

```

```

    for (i = 0; i < n; i++) {

```

```

        if (0 < i & (p[i].qno == 1 & p[i].arrival <=

```

```

            count_time & p[i].remaining
            > 0) {

```

```

                executed = i;

```

```

            }

```

```

            if (p[i].remaining > tq) {

```

```

                p[i].remaining -= tq;

```

```

                count_time += tq;
            }

```

```

        }

```

```

    } else {

```

```

        count_time += p[i].remaining;

```

```

        p[i].remaining = 0;

```

```

        p[i].completion = count_time;

```

```

        p[i].turnaround = p[i].completion

```

```

        - p[i].arrival;

```

```

        p[i].waiting = p[i].turnaround - p[i].
        burst;

```

```

        if (0 < p[i].waiting)

```



```

total-tat += p[i].turnaround;
total-wt += p[i].waiting;
completed++;
}
}

if (executed) {
    continue;
}

int min_arrival = 1e9;
for (i = 0; i < m; i++) {
    if (p[i].qno == 2 && p[i].arrival
        <= current_time &&
        p[i].remaining > 0) {
        if (p[i].arrival < min_arrival) {
            min_arrival = p[i].arrival;
            idx = i;
        }
    }
}

if (idx != -1) {
    executed = 1;
    int run-time = p[idx].remaining;
    for (int t = 1; t <= run-time; t++) {
        current_time++;
        p[idx].remaining--;
    }
    int sys_arrival = 0;
    for (i = 0; i < n; i++) {
        if (p[i].qno == 1 && p[i].arrival
            <= current_time && p[i].remaining > 0) {

```



```

        sys-arrind = 1
        break;
    }

    if (sys-arrind)
        break;
    if (p[idx].remaining == 0) {
        p[idx].completion = current-time;
        p[idx].turnaround = p[idx].completion
            - p[idx].arrival;
        p[idx].waiting = p[idx].turnaround
            - p[idx].burst;
        total-tat += p[idx].turnaround;
        total-wt += p[idx].waiting;
        computed++;
    }
}

printf("In PIDLE @ %d AT %d BT %d ET %d AT %d WT\n",
        itwt, n);
for (i=0; i<n; i++) {
    printf("%d %d %d %d %d %d %d %d %d %d\n",
           p[i].pid, p[i].qno, p[i].arrival, p[i].burst,
           p[i].completion, p[i].turnaround,
           p[i].waiting);
}

printf("Average TAT = %.2f", total-tat/n);
printf("Average wt = %.2f", total-wt/n);
return 0;
}

```


Output

Enter number of Process: 4

P1 AT: 0

P1 BT: 4

Queue: 1

P2 AT: 0

P2 BT: 3

Queue: 2

P3 AT: 0

P3 BT: 8

Queue: 2

P4 AT: 10

P4 BT: 5

Queue: 1

Time Quantum: 2

Proc	Queue	AT	BT	CT	TAT	WT
P1	1	0	4	6	6	6
P2	1	0	3	7	7	7
P3	2	0	8	20	20	20
P4	1	10	5	15	15	5

Average TAT = 9.50

Average WT = 4.50

Program - 3

Output:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

CODE

=>Rate Monotonic Scheduling

```
#include
<stdio.h> int
main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu
    Sharan\n\n"); int n, i, j;

    printf("Enter number of tasks:
    "); scanf("%d", &n);

    int bt[n], period[n];
    printf("Enter Burst Time and
    Period:\n");
    for(i = 0; i < n; i++){
        scanf("%d %d", &bt[i], &period[i]);
    }
    for(i = 0; i < n - 1; i++){
        for(j = i + 1; j < n; j++){
            if(period[i] > period[j]){
                int temp;

                temp = period[i];
                period[i] =
                period[j];
                period[j] = temp;
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
}
```



```

printf("\nTask\tBurst Time\tPeriod\n");

for(i = 0; i < n; i++)
{
    printf("T%d\t%d\t\t", i + 1,
        bt[i],
        period[i]);
}

return 0;
}

```

Result

USN : 1WA24CS223
 Name: Priyanshu Sharan

Enter number of tasks: 4
 Enter Burst Time and Period:
 2 10
 1 5
 3 20
 2 8

Task	Burst Time	Period
T1	1	5
T2	2	8
T3	2	10
T4	3	20

priyanshusharan@Priyanshus-MacBook-Air os-coding %

Q

Write a c-program to simulate Real Time CPU Scheduling algorithm.

- a) Rate Monotonic
- b) Earliest Deadline First
- c) Proportional Scheduling

a) // RATE MONOTONIC SCHEDULING

```
#include <stdio.h>
#include <math.h>
```

```
int gcd(int a, int b) {
    if (b == 0)
        return a;
    return gcd(b, a % b);
}
```

```
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
```

```
int main() {
```

```
    int n;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
```

```
    int burst[20], period[20], rem_burst[20];

    printf("Enter the time periods: \n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &period[i]);
        rem_burst[i] = 0;
```

```
    }
```

```
    printf("Enter the time periods: \n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &period[i]);
```

```
        rem_burst[i] = 0;
```



```

    }
    }
    int highest-priority-process = -1;
    int min-period = ...
    for (int i=0; i<n; i++) {
        if (rem-burst[i] > 0 && period[i] <
            min-period) {
            mid-period = period[i];
            highest-priority-process = i;
        }
    }
    if (highest-priority-process !=
        current-process) {
        if (highest-priority-process == -1) {
            printf("x.d's on cards:
            CPU is idle\n", t);
        }
        else {
            printf("x.d's on cards: Process
            x.d running\n", t, highest-priority-
            process + 1);
            current-process = highest-priority-process;
        }
    }
    if (current-process != -1) {
        rem-burst[current-process]--;
    }
    }
    return 0;
}

```


Output

1wA24CS223

Enter the number of processes: 2

Enter the CPU burst times in ms:

20 35

Enter the time period: 100

50 100

LCM = 100

Rate Monotonic Scheduling

PID Burst Period

1 20 50

2 35 100

$0.750000 \leq 0.828427 \Rightarrow \text{true}$

Scheduling occurs for 100ms

0ms onwards: Process 1 running

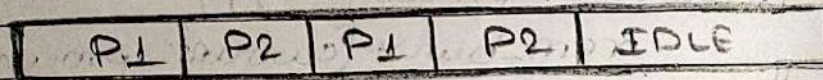
20ms onwards: Process 2 running

50ms onwards: Process 1 running

70ms onwards: Process 2 running

75ms onwards: CPU is idle

- Grant Chart -



0 20 50 70 75 100

=> Earliest Deadline First

```
#include <stdio.h>
int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int n, i, j;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    int bt[n], deadline[n];
    printf("Enter Burst Time and Deadline:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d %d", &bt[i], &deadline[i]);
    }

    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(deadline[i] > deadline[j])
            {
                int temp;

                temp = deadline[i];
                deadline[i] = deadline[j];
                deadline[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
}
```



```

printf("\nTask\tBurst Time\tDeadline\n");
for(i = 0; i < n; i++)
{
    printf("T%d\t%d\t\t%d\n",
           i + 1,
           bt[i],
           deadline[i]);
}

return 0;
}

```

Result

```

USN : 1WA24CS223
Name: Priyanshu Sharan

```

```

Enter number of tasks: 4
Enter Burst Time and Deadline:
2 12
1 5
3 20
2 8

```

Task	Burst Time	Deadline
T1	1	5
T2	2	8
T3	2	12
T4	3	20

```

) priyanshusharan@Priyanshus-MacBook-Air os-coding % 

```


b) // Earliest Deadline First

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter the number of processes\n");
    scanf("%d", &n);
    int arrival[n+1], burst[n+1], deadline[n+1], remaining[n+1];
    for (int i = 1; i <= n; i++)
    {
        printf("Enter Arrival time, Burst time and Deadline for Task %d: ", i);
        scanf("%d %d %d", &arrival[i], &burst[i], &deadline[i]);
        remaining[i] = burst[i];
    }

    int time = 0;
    int computed = 0;
    printf("\nOutput:\n");
    while (computed < n)
    {
        int min_deadline = 999999;
        int selected = -1;
        for (int i = 1; i <= n; i++)
        {
            if (arrival[i] <= time
                && remaining[i] > 0)
            {
                if (deadline[i] < min_deadline)
                {
                    min_deadline = deadline[i];
                    selected = i;
                }
            }
        }
    }
}
```



```

    }
    if (selected == -1) {
        printf("%dms: CPU is idle.\n",
            time);
        time = time + 1;
    }
    else {
        printf("%dms: Task %d is
            running.\n", time, selected);
        remaining[selected] =
            remaining[selected] - 1;
        time = time + 1;
        if (remaining[selected] == 0) {
            completed = completed + 1;
        }
    }
}

printf("Scheduling complete at %dms.\n",
    time);
return 0;
}

```

Output

Output

Enter the number of processes: 3

Enter Arrival time, Burst time, and

Deadline for Task 1: 0 3 7

Enter Arrival time, Burst time, and

Deadline for Task 2: 0 2 4

Enter Arrival time, Burst time, and

Deadline for Task 3: 0 2 8

Output

0ms : Task 2 is running

1ms : Task 2 is running

2ms : Task 1 is running

3ms : Task 1 is running

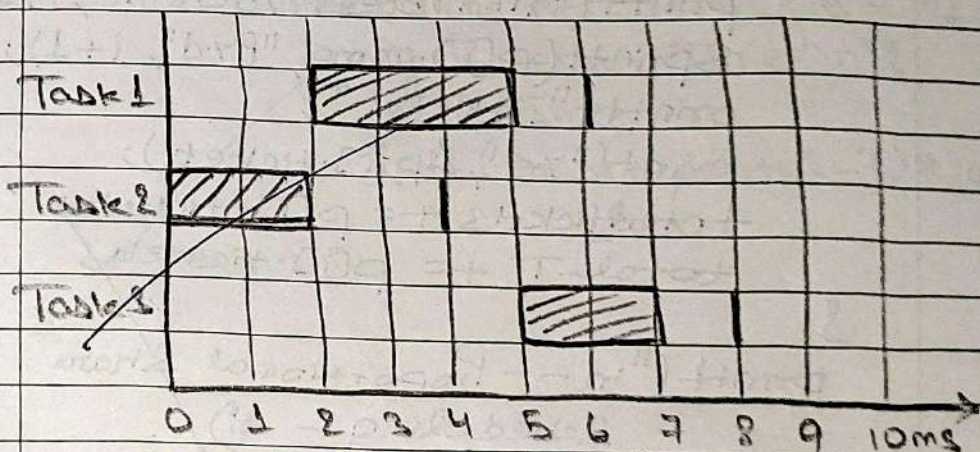
4ms : Task 1 is running

5ms : Task 3 is running

6ms : Task 3 is running

Scheduling compute at 7ms

Gantt Chart



Task 1 Burst 3ms

T1 Deadline 7ms

Task 2 Burst 2ms

T2 Deadline 4ms

Task 3 Burst 2ms

T3 Deadline 8ms

=> Proportional Scheduling

```
#include <stdio.h>
int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int n, i;
    float total = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int share[n];
    printf("Enter CPU Shares:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &share[i]);
        total += share[i];
    }
    printf("\nProcess\tCPU Share(%%)\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%.2f%%\n",
               i + 1,
               (share[i] * 100.0) / total);
    }

    return 0;
}
```

Result

USN : 1WA24CS223

Name: Priyanshu Sharan

Enter number of processes: 5

Enter CPU Shares:

10 15 25 20 30

Process	CPU Share(%)
---------	--------------

P1	10.00%
----	--------

P2	15.00%
----	--------

P3	25.00%
----	--------

P4	20.00%
----	--------

P5	30.00%
----	--------

› priyanshusharan@Priyanshus-MacBook-Air os-coding %

C) Proportional Share Scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
typedef struct {
    char name[5];
    int tickets;
} Process;
```

```
int main() {
    int n, total_tickets = 0;
    float total_T = 0.0;
    printf("Enter the number of Processes: ");
    scanf("%d", &n);
    Process p[n];
    srand(time(NULL));
    for(int i=0; i<n; i++) {
        printf("\n Process %d: \n", i+1);
        sprintf(p[i].name, "P%d", i+1);
        printf("Tickets: ");
        scanf("%d", &p[i].tickets);
        total_tickets += p[i].tickets;
        total_T += p[i].tickets;
    }
    printf("\n -- Proportional Share Scheduling - ti");

    int m;
    printf("Enter the Time Period for Scheduling: ");
```



```

scanf("%d", &m);
for (int i=0; i<m; i++) {
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index = 0;
    for (int j=0; j<n; j++) {
        accumulated_tickets += p[j].tickets;
        if (winning_ticket <= accumulated_tickets) {
            winning_index = j;
            break;
        }
    }
    printf("Tickets Picked: %d Winner: %s\n", winning_ticket, p[winner_index].name);
}
for (int i=0; i<n; i++) {
    printf("In the Process: %s gets %.2f % of Processor Time\n", p[i].name, (p[i].tickets / total_T) * 100);
}
return 0
}

```


Sample Output

Enter the number of Processes: 3

Process 1:

Tickets: 50

Process 2:

Tickets: 20

-- Proportional Share Scheduling --

Enter the Time Period for scheduling: 5

Tickets picked: 14, Winner: P1

Tickets picked: 8, Winner: P3

Tickets picked: 6, Winner: P2

Tickets picked: 4, Winner: P1

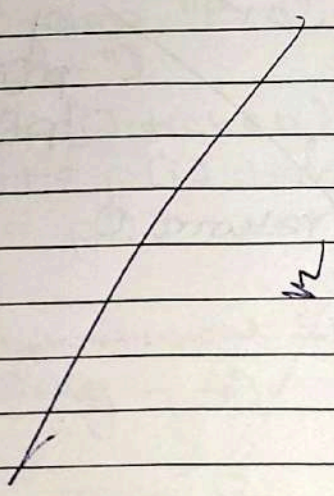
Tickets picked: 9, Winner: P1

Result:

The Process P1 gets 50.00% of Processing Time

The Process P2 gets 30.00% of Processing Time

The Process P3 gets 20.00% of Processing Time



Program - 4

Question:

Write a C program to simulate producer-consumer problem using semaphores

CODE

```
#include <stdio.h>
int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int mutex = 1;
    int full = 0;
    int empty = 5;
    int x = 0;
    int choice;

    while(1)

    {
        printf("\n1. Produce");
        printf("\n2. Consume");
        printf("\n3. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                if((mutex == 1) && (empty != 0))
                {
                    mutex =
                    0;
                    full++;
                    empty--;
                    x++;
                    printf("Producer produces item %d\n", x);
                    mutex = 1;
                }
                else
```



```

        {
            printf("Buffer is Full!\n");
        }
        break;

    case 2:
        if((mutex == 1) && (full != 0))
        {
            mutex = 0;
            printf("Consumer consumes item %d\n",
                x); full--;
            empty++;
            ; x--;
            mutex = 1;
        }
        else
        {
            printf("Buffer is Empty!\n");
        }
        break;

    case 3:
        return 0;

    default:
        printf("Invalid Choice\n");
    }
}

return 0;
}

```

Result

USN : 1WA24CS223

Name: Priyanshu Sharan

1. Produce

2. Consume

3. Exit

Enter your choice: 1

Producer produces item 1

1. Produce

2. Consume

3. Exit

Enter your choice: 1

Producer produces item 2

1. Produce

2. Consume

3. Exit

Enter your choice: 2

Consumer consumes item 2

1. Produce

2. Consume

3. Exit

Enter your choice: 1

Producer produces item 2

1. Produce

2. Consume

3. Exit

Enter your choice: 3

priyanshusharan@Priyanshus-MacBook-Air os-coding %

LAB PROGRAM : Write a C-program to
Simulate

a) Producer Consumer Problem using Semaphores

Bounded - Buffer Problem

Pseudo Code

Shared data

Semaphore mutex initialized to value 1

Semaphore full initialized to value 0

Semaphore empty initialized to value n

// Empty $\rightarrow$ counts number of empty buffers

// Full $\rightarrow$ count number of full buffers

// Producer Process

do {

 /* produce an item in next-produced */

 wait (empty);

 wait (mutex);

 /* add new producer to the buffer */

 Signal (mutex);

 Signal (full);

} while (true);

// Consumer Process


```

do {
    wait (full);
    wait (mutex);
    /* remove an item from buffer to
       next = consume */
    ...
    signal (mutex);
    signal (mutex);
    ...
    /* consume the item in next = consume */
    ...
} while (true);

```

// case II

0 a
1 b
2 c
:
9

empty = 5

full = 3

Producer

wait (empty) = 4

preempt / context Substituting to
consumer

Consumer

wait(full) = 3

wait(mutex)

// CSP

next_consumed = Butt (out)

out = (out + 1) / n

// context switch to Producer

i.e wait(mutex) from 0 to 1, hence

blocked, and control comes to consumer

signal(mutex)

✓
even

BOUNDED BUFFER (Producer Consumer)

```
#include <stdio.h>
#include <stdlib.h>
int stack[100], top = -1, max;
int itemCounter = 0;
void produce() {
    if (top == max - 1) {
        printf("Buffer is full ! \n");
        return;
    }
    itemCounter++;
    stack[++top] = itemCounter;
    printf("Producer has produced: item %d \n",
        itemCounter);
}
void consume() {
    if (top == -1) {
        printf("Buffer is empty \n");
        return;
    }
    printf("Consumer has consume : item %d \n",
        stack[top--]);
}
int main() {
    int choice;
    printf("Enter the size of the buffer: ");
    scanf("%d", &max);
    if (max > 100) {
        printf("Max size is 100. \n");
        return 0;
    }
}
```



```
printf("\nEnter 1: Producer 1: 2: Consumer 3: Exit\n");
```

```
while (1){
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d", &choice);
```

```
    switch (choice){
```

```
        case 1:
```

```
            produce();
```

```
            break;
```

```
        case 2:
```

```
            consume();
```

```
            break;
```

```
        case 3:
```

```
            printf("Exiting program...\n");
```

```
            exit(0);
```

```
        default:
```

```
            printf("Invalid choice!\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Output

Enter the size of buffer: 2

Enter

1. Producer

2. Consumer

3. Exit

Enter your choice: 1

Producer has produced: item 1

Enter your choice: 1

Producer has produced Item 2.

Enter your choice: 1

Button 0 is full!

Enter your choice: 2

Consumer has consumed:- item 2

Enter your choice: 12

Consumer has consumed: item 1

Enter your choice: 2

Butter is empty.

Enter your choice: 3

Exiting Program :-

["of..."]

(d) Fine

12/10/2012

(*Malvastrum*) *Malvastrum*

U. m. 1977

2/12

5th method for using with interest

1948

November 1

1. *Staphylococcus aureus* . S.

22-11-5

1. 1910 1911 1912 1913 1914 1915 1916 1917 1918 1919 1920 1921 1922 1923 1924 1925 1926 1927 1928 1929 1930 1931 1932 1933 1934 1935 1936 1937 1938 1939 1940 1941 1942 1943 1944 1945 1946 1947 1948 1949 1950 1951 1952 1953 1954 1955 1956 1957 1958 1959 1960 1961 1962 1963 1964 1965 1966 1967 1968 1969 1970 1971 1972 1973 1974 1975 1976 1977 1978 1979 1980 1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007 2008 2009 2010 2011 2012 2013 2014 2015 2016 2017 2018 2019 2020 2021 2022 2023 2024 2025 2026 2027 2028 2029 2030 2031 2032 2033 2034 2035 2036 2037 2038 2039 2040 2041 2042 2043 2044 2045 2046 2047 2048 2049 2050 2051 2052 2053 2054 2055 2056 2057 2058 2059 2060 2061 2062 2063 2064 2065 2066 2067 2068 2069 2070 2071 2072 2073 2074 2075 2076 2077 2078 2079 2080 2081 2082 2083 2084 2085 2086 2087 2088 2089 2090 2091 2092 2093 2094 2095 2096 2097 2098 2099 2100 2101 2102 2103 2104 2105 2106 2107 2108 2109 2110 2111 2112 2113 2114 2115 2116 2117 2118 2119 2120 2121 2122 2123 2124 2125 2126 2127 2128 2129 2130 2131 2132 2133 2134 2135 2136 2137 2138 2139 2140 2141 2142 2143 2144 2145 2146 2147 2148 2149 2150 2151 2152 2153 2154 2155 2156 2157 2158 2159 2160 2161 2162 2163 2164 2165 2166 2167 2168 2169 2170 2171 2172 2173 2174 2175 2176 2177 2178 2179 2180 2181 2182 2183 2184 2185 2186 2187 2188 2189 2190 2191 2192 2193 2194 2195 2196 2197 2198 2199 2200 2201 2202 2203 2204 2205 2206 2207 2208 2209 2210 2211 2212 2213 2214 2215 2216 2217 2218 2219 2220 2221 2222 2223 2224 2225 2226 2227 2228 2229 2230 2231 2232 2233 2234 2235 2236 2237 2238 2239 2240 2241 2242 2243 2244 2245 2246 2247 2248 2249 2250 2251 2252 2253 2254 2255 2256 2257 2258 2259 2260 2261 2262 2263 2264 2265 2266 2267 2268 2269 2270 2271 2272 2273 2274 2275 2276 2277 2278 2279 2280 2281 2282 2283 2284 2285 2286 2287 2288 2289 2290 2291 2292 2293 2294 2295 2296 2297 2298 2299 2300 2301 2302 2303 2304 2305 2306 2307 2308 2309 2310 2311 2312 2313 2314 2315 2316 2317 2318 2319 2320 2321 2322 2323 2324 2325 2326 2327 2328 2329 2330 2331 2332 2333 2334 2335 2336 2337 2338 2339 2340 2341 2342 2343 2344 2345 2346 2347 2348 2349 2350 2351 2352 2353 2354 2355 2356 2357 2358 2359 2360 2361 2362 2363 2364 2365 2366 2367 2368 2369 2370 2371 2372 2373 2374 2375 2376 2377 2378 2379 2380 2381 2382 2383 2384 2385 2386 2387 2388 2389 2390 2391 2392 2393 2394 2395 2396 2397 2398 2399 2400 2401 2402 2403 2404 2405 2406 2407 2408 2409 2410 2411 2412 2413 2414 2415 2416 2417 2418 2419 2420 2421 2422 2423 2424 2425 2426 2427 2428 2429 2430 2431 2432 2433 2434 2435 2436 2437 2438 2439 2440 2441 2442 2443 2444 2445 2446 2447 2448 2449 2450 2451 2452 2453 2454 2455 2456 2457 2458 2459 2460 2461 2462 2463 2464 2465 2466 2467 2468 2469 2470 2471 2472 2473 2474 2475 2476 2477 2478 2479 2480 2481 2482 2483 2484 2485 2486 2487 2488 2489 2490 2491 2492 2493 2494 2495 2496 2497 2498 2499 2500 2501 2502 2503 2504 2505 2506 2507 2508 2509 2510 2511 2512 2513 2514 2515 2516 2517 2518 2519 2520 2521 2522 2523 2524 2525 2526 2527 2528 2529 2530 2531 2532 2533 2534 2535 2536 2537 2538 2539 2540 2541 2542 2543 2544 2545 2546 2547 2548 2549 2550 2551 2552 2553 2554 2555 2556 2557 2558 2559 2560 2561 2562 2563 2564 2565 2566 2567 2568 2569 2570 2571 2572 2573 2574 2575 2576 2577 2578 2579 2580 2581 2582 2583 2584 2585 2586 2587 2588 2589 2590 2591 2592 2593 2594 2595 2596 2597 2598 2599 2600 2601 2602 2603 2604 2605 2606 2607 2608 2609 2610 2611 2612 2613 2614 2615 2616 2617 2618 2619 2620 2621 2622 2623 2624 2625 2626 2627 2628 2629 2630 2631 2632 2633 2634 2635 2636 2637 2638 2639 2640 2641 2642 2643 2644 2645 2646 2647 2648 2649 2650 2651 2652 2653 2654 2655 2656 2657 2658 2659 2660 2661 2662 2663 2664 2665 2666 2667 2668 2669 2670 2671 2672 2673 2674 2675 2676 2677 2678 2679 2680 2681 2682 2683 2684 2685 2686 2687 2688 2689 2690 2691 2692 2693 2694 2695 2696 2697 2698 2699 2700 2701 2702 2703 2704 2705 2706 2707 2708 2709 2710 2711 2712 2713 2714 2715 2716 2717 2718 2719 2720 2721 2722 2723 2724 2725 2726 2727 2

1000. 1000. 1000. 1000. 1000.

1. 1910's and 1920's

Program - 4

Question:

Write a C program to simulate the concept of Dining Philosophers problem

CODE

```
#include <stdio.h>
int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int n = 5;

    for(int i = 0; i < n; i++)
    {
        printf("Philosopher %d is Thinking\n", i + 1);

        printf("Philosopher %d takes Fork %d and Fork %d\n",
            i + 1,
            i + 1,
            (i + 1) % n + 1);

        printf("Philosopher %d is Eating\n", i + 1);

        printf("Philosopher %d puts down Fork %d and Fork %d\n\n",
            i + 1,
            i + 1,
            (i + 1) % n + 1);
    }

    return 0;
}
```

Result

USN : 1WA24CS223

Name: Priyanshu Sharan

Philosopher 1 is Thinking
Philosopher 1 takes Fork 1 and Fork 2
Philosopher 1 is Eating
Philosopher 1 puts down Fork 1 and Fork 2

Philosopher 2 is Thinking
Philosopher 2 takes Fork 2 and Fork 3
Philosopher 2 is Eating
Philosopher 2 puts down Fork 2 and Fork 3

Philosopher 3 is Thinking
Philosopher 3 takes Fork 3 and Fork 4
Philosopher 3 is Eating
Philosopher 3 puts down Fork 3 and Fork 4

Philosopher 4 is Thinking
Philosopher 4 takes Fork 4 and Fork 5
Philosopher 4 is Eating
Philosopher 4 puts down Fork 4 and Fork 5

Philosopher 5 is Thinking
Philosopher 5 takes Fork 5 and Fork 1
Philosopher 5 is Eating
Philosopher 5 puts down Fork 5 and Fork 1

priyanshusharan@Priyanshus-MacBook-Air os-coding %

Simulate Dining Philosophers Problem

Budoi (philosophers) $\leftarrow 10$

Shared Data $\leftarrow 10$

Semaphore chopstick[5]

Semaphore (chopstick) $\leftarrow 02$

Initialization

Chopstick [5] = {1, 1, 1, 1, 1} 02

// Structure of Philosopher

do { (chopstick) $\leftarrow 02$

wait(chopstick[i]);

wait(chopstick[(i+1)%5]);

// eat

signal(chopstick[i]);

signal(chopstick[(i+1)%5]);

// think

} while (true)

02 (chopstick) $\leftarrow 02$

TRACING

S0 - 1

10, 02 $\leftarrow$ 09

S1 - 1

02, 10 $\leftarrow$ 19

S2 - 1

20, 02 $\leftarrow$ 29

S3 - 1

02, 20 $\leftarrow$ 39

S4 - 1

02, 02 $\leftarrow$ 49

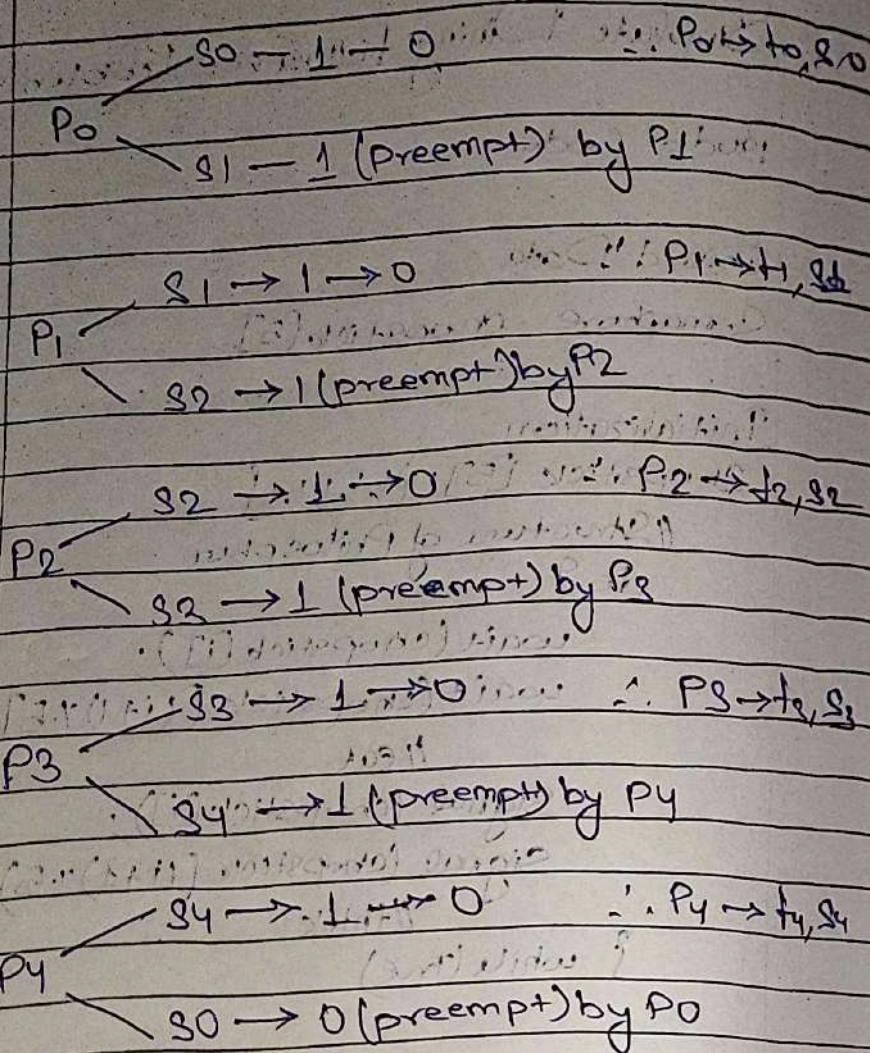
19, 02 $\leftarrow$ 59

02, 02 $\leftarrow$ 69

02, 02

02, 02

02, 02



The above condition leads to Deadlock
Solution for Deadlock

$P_0 \rightarrow S_0, S_1$
 $P_1 \rightarrow S_1, S_2$
 $P_2 \rightarrow S_2, S_3$
 $P_3 \rightarrow S_3, S_4$
 $P_4 \rightarrow S_4, S_0$

exchange
 $P_4 \rightarrow S_0, S_4$
 $\downarrow \quad \downarrow$
 wait(0)
 $\rightarrow$ blocked

P₃ → S₃ S₄

↓ ↓

wait(1) wait(1)

↓ ↓
0 0

Enter critical section

eat()

pushfork(lett) & pushfork(right)

P₂ → S₂, S₃

↓ ↓

↑ ↑
wait(1) wait(1)

↓ ↓
0 0

Enters critical section, eat

pushfork(lett) & pushfork(right)

return P₁ → S₁, S₂

↓ ↓

↑ ↑
wait(1) wait(1)

↓ ↓
0 0

Enters critical section, eat then

pushfork(lett) & pushfork(right)

P₀ → S₀ S₁

↓ ↓

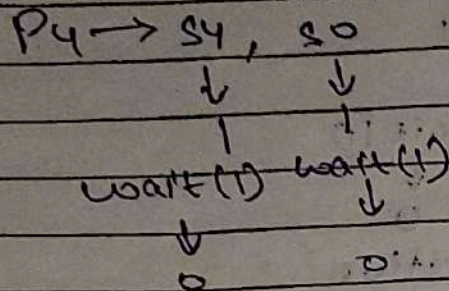
↑ ↑
wait(1) wait(1)

↓ ↓
0 0

Enters critical section, eat, then

pushback(lett) & pushback(right)

S₀ → 1 S₁ → 1



Enter critical section eat them
 push back (left) and push back (right)
 $S_4 \rightarrow 1$ & $S_0 \rightarrow 1$

C - CODE

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5 // Number of philosophers
pthread_mutex_t forks[N]; // One mutex per lock

pthread_t philosophers[N];
void* philosopher(void* num)
{
    int id = *(int*) num;
    int left = id; // left fork index
    int right = (id+1) % N // right fork index
    while (1) {
        printf("Philosopher %d is thinking\n", id);
        sleep(1); // Thinking
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up", id);

```



```

    left fork %d\n", id, left);
    pthread_mutex_lock(&forks[left]);
    printf("Philosopher %d picked up\n", id);
    right fork %d\n", id, right);
}
else {

```

```

    pthread_mutex_lock(&forks[right]);
    printf("Philosopher %d picked up\n", id);
    right fork %d\n", id, right);
    fork pthread_mutex_lock(&forks[left]);
    printf("Philosopher %d picked up\n", id);
    left fork %d\n", id, left);
}

```

```

    printf("Philosopher %d is eating\n", id);
    sleep(2);

```

```

    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);
    printf("Philosopher %d put down forks\n", id, left, right);
}

```

```

    return NULL;
}

```

```

int main() {

```

```

    int i;

```

```

    int ids[N];

```

```

    for(i=0; i<N; i++) {

```

```

        pthread_mutex_init(&forks[i], NULL);

```

```

        ids[i] = i;
    }

```

```

    for(i=0; i<N; i++) {

```

```

        pthread_create(&philosopher[i],

```

```

            NULL, philosopher, &ids[i]);
    }
}

```



```

}
for(i=0; i<N; i++) {
    pthread_join(philosophers[i], NULL);
}
for(i=0; i<N; i++) {
    pthread_mutex_destroy(&forkes[i]);
}
return 0;
}

```

① OUTPUT

Philosopher 0 is thinking
Philosopher 3 is thinking

24/4

Program - 5

Question:

Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

CODE

```
#include <stdio.h>
int main()
{
    printf("USN : 1WA24CS223\n");
    printf("Name: Priyanshu Sharan\n\n");

    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Maximum Matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources:\n");
    for(int i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
```

```

int finish[n];
int safeSeq[n];

for(int i = 0; i < n; i++)
    finish[i] = 0;
int count = 0;

while(count < n)

{
    int found = 0;

    for(int i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int j;

            for(j = 0; j < m; j++)
            {
                if(need[i][j] > avail[j])
                    break;
            }

            if(j == m)
            {
                for(int k = 0; k < m; k++)
                    avail[k] += alloc[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
    {
        printf("\nSystem is NOT in Safe State\n");
        return 0;
    }
}

```



```

    }

    printf("\nSystem is in Safe State\n");
    printf("Safe Sequence: ");

    for(int i = 0; i < n; i++)
        printf("P%d ", safeSeq[i]);

    printf("\n");

    return 0;
}

```

Result

USN : 1WA24CS223

Name: Priyanshu Sharan

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources:

3 3 2

System is in Safe State

Safe Sequence: P1 P3 P4 P0 P2

) priyanshusharan@Priyanshus-MacBook-Air os-coding %

PROGRAM: BANKER'S ALGORITHM

PART A Safety Algorithm (Allocation, Request, Available)

Algorithm:

1) Let work & Finish be vectors of length m and n

initialize work = Available

Finish[i] = false, for $i = 0$ to $i = n$

2) Find an index i such that

a) Finish[i] = false

b) Need[i] $\leq$ work

3) work = work + Allocation

Finish[i] = true

goto step 2

4) if Finish[i] == true for all i

then system is safe

Eg Determine whether the system is in safe state or not using safety algorithm

	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	3	3	2	7	4	3
P ₁	2	0	0	3	2	2				1	2	2
P ₂	3	0	2	9	0	2				6	0	0
P ₃	2	1	1	2	2	2				0	1	1
P ₄	0	0	2	4	3	3				4	3	1

Tracing

1 work = 3 3 2
 finish[i] = false i = 0 to 4
 P0 → not sat

2 finish[0] = false
~~7 4 3 ≤ 3 3 2 (false)~~
 Need < work
 7 4 3 ≤ 3 3 2 (false)
 finish[0] = true
 P0 → sat

3 finish[1] = false
 Need < work
 1 1 2 ≤ 3 3 2 (true)
 work = 3 3 2 + 2 0 0
 = 5 3 2
 P1 → sat

finish[2] = false
 Need < work
 6 0 0 ≤ 5 3 2 (false)
 P2 → not sat

finish[3] = true
 finish[4] = false
 Need < work
 0 1 1 ≤ 5 3 2 (true)
 P3 → sat

work = 5 3 2 + 2 1 0
 = 7 4 3
 P4 → sat

finish[3] = true
 finish[4] = true
 P4 → sat

finish [4] = false

Need 5 work

431 < 743 (true)

$$\therefore \text{work} = 743 + 002 \\ = 745$$

finish [0] = false

Need 5 work

743 < 745 (true)

~~work > 74~~

$$\text{work} = 745 + 010 \\ = 755$$

finish [2] = false

Need 5 work

000 < 755 (true)

$$\text{work} = 755 + 302 \\ = 1057$$

finish [2] = true

Sati Sequence

P1, P3, P4, P0, P2

C-CODE

```
#include <stdio.h>
int main() {
    int n, m, i, j, k;
    printf("Enter number of processes: ")
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];
```

```
    printf("Enter Allocation Matrix: In");
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
```

```
    printf("Enter Maximum Demand Matrix: In");
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            scanf("%d", &max[i][j]);
        }
    }
```

```
    printf("Enter Available Resou: In");
    for(i=0; i<m; i++) {
        scanf("%d", &avail[i]);
    }
```



```

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        need[i][j] = min(need[i][j], avail[i])
    }
}

```

```

int finish[n], startSeg[n]
int work[m];

```

```

for (i=0; i<n; i++) {
    finish[i] = 0;
}

```

```

for (i=0; i<m; i++) {
    work[i] = 0;
    work[i] = avail[i]
}

```

```

int count = 0;

```

```

while (count < n) {

```

```

    if (found == 0)

```

```

        for (i=0; i<n; i++) {
            if (finish[i] == 0) {

```

```

                cout << " ";
                int flag = 1;

```

```

                for (j=0; j<m; j++) {

```

```

                    if (need[i][j] > work[j])

```

```

                        flag = 0;

```

```

                        break;

```

```

                }
            }
        }
    }
}

```



```

if (flag == 1) {
    for (k = 0; k < m; k++) {
        word[k] += alloc[i][k]
    }
}

```

```

stateSeq[count++] = i;
finish[i] = 1;
found = 1;
}
}

```

```

}
}

```

```

if (found == 0) {
    break;
}

```

```

}

```

```

if (count == n) {

```

```

    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

```

```

    for (i = 0; i < n; i++) {
        printf("P%d", stateSeq[i]);

```

```

        if (i != n - 1)

```

```

            printf(" -> ");

```

```

    }
    else {

```

```

        printf("\nSystem is NOT in  
a safe state.\n");

```

```

    }

```

```

    return 0;
}

```


SAMRE OUTPUT

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Demand Matrix

7 5 3

3 2 2

9 0 2

2 2 2

0 4 3 3 3

0 0 0 0 0

Enter Available Resource

3 3 2

System is in a safe sequence

Safe Sequence is: $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

=> Deadlock Detection
CODE

```
#include <stdio.h>

int main(){
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);
    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];
    printf("\nEnter Allocation Matrix:\n");
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("\nEnter Maximum Matrix:\n");
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            {
                scanf("%d", &max[i][j]);
            }
        }
    }
    printf("\nEnter Available Resources:\n");
    for(int i = 0; i < m; i++){
        scanf("%d", &avail[i]);
    }
    for(int i = 0; i < n; i++){
```

```

        for(int j = 0; j < m; j++){
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    int finish[n];
    int safe[n];
    for(int i = 0; i < n; i++)
        finish[i] = 0;
    int count = 0;
    while(count < n){
        int found = 0;
        for(int i = 0; i < n; i++){
            if(finish[i] == 0){
                int possible = 1;
                for(int j = 0; j < m; j++){
                    if(need[i][j] > avail[j]){
                        possible = 0;
                        break;
                    }
                }
                if(possible){
                    for(int j = 0; j < m; j++){
                        avail[j] += alloc[i][j];
                    }
                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
    }

```



```
    }

    if(found == 0)
        break;
}

if(count == n){
    printf("\nSystem is in Safe State\n");
    printf("Safe Sequence: ");

    for(int i = 0; i < n; i++){
        printf("P%d", safe[i]);

        if(i != n - 1)
            printf(" -> ");
    }
}

else{
    printf("\nSystem is NOT in Safe State\n");
}

return 0;
}
```

Result

```
1WA24CS223
Priyanshu Sharan
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in Safe State
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2%
priyanshusharan@Priyanshus-MacBook-Air os-coding %
```


ART B DEADLOCK DETECTION

Deadlock-detection (Allocation, Request, Available)

a) $work \leftarrow Available$
 b) for $i \leftarrow 1$ to n do
 if $allocation[i] = 0$ then
 $finish[i] \leftarrow true$
 else
 $finish[i] \leftarrow false$

End if

End for

c) Repeat:

$found \leftarrow false$

 for $i \leftarrow 1$ to n do

 if ($finish[i] = false$ AND

$Request[i] \leq work$) Then

$work \leftarrow work + Allocation[i]$

$Finish[i] \leftarrow true$

 print

$found \leftarrow true$

 end if

 end for

 until $found = true$

d) If all $finish[i] = true$ then

 No Deadlock

Else

 Deadlock exists

Eg	Process	Allocation			Request		
		A	B	C	A	B	C
	P ₀	0	1	0	0	0	0
	P ₁	2	0	0	2	0	2
	P ₂	3	0	3	0	0	0
	P ₃	2	1	1	1	0	0
	P ₄	0	0	2	0	0	2

Available = (0, 0, 0)

Check P₀

Step 1: Request (P₀) = 0 0 0

(0 0 0) ≤ (0 0 0) - (true)

P₀ → first

New Avail = 0 0 0 + 0 1 0
= 0 1 0

Step 2: Request (P₁) ≤ Avail

2 0 0 ≤ 0 1 0 (No)

for next Request (P₂) ≤ Avail

0 0 0 ≤ 0 1 0 (true)

P₂ → second

New Avail = 0 1 0 + 3 0 3
= 3 1 3

Step 3: Request (P₃) ≤ Avail

2 1 1 ≤ 3 1 3 (true)

P₃ → Third

New Avail = 3 1 3 + 2 1 1
= 5 2 4

Step 4: Request (P₄) ≤ Avail

0 0 2 ≤ 5 2 4 (true)

P₄ → fourth

Step 6:- Request(P1) $\leq$ Avail
 $2 \ 0 \ 2 \leq 3 \ 2 \ 6$ (true)
 $P1 \rightarrow \text{finish}$

New Avail = $3 \ 2 \ 6 + 2 \ 0 \ 0$
 $= 5 \ 2 \ 6$

No Dead Lock

Exit

$P0 \rightarrow P2 \rightarrow P3 \rightarrow P4 \rightarrow P1$

Q-CODE

```
#include <stdio.h>
```

```
int main() {
```

```
int n, m, i, j, k;
```

```
printf("Enter the number of  
processes: ");
```

```
scanf("%d", &n);
```

```
printf("Enter the number of  
resources: ");
```

```
scanf("%d", &m);
```

```
int alloc[n][m], request[n][m],  
avail[m];
```

```
int finish[n], safeSeq[n];
```

```
printf("Enter Allocation matrix: \n")
```



```

for(i=0; i<n; i++) {
    print("Process: ", i);
    for(j=0; j<m; j++) {
        scan("%d", &avail[j]);
    }
}

```

```

printt("Enter the request matrix:");
for(i=0; i<n; i++) {
    printt("Process %d: ", i);
    for(j=0; j<m; j++) {
        scan("%d", &request[i][j]);
    }
}

```

```

printt("Enter the available resources");
for(i=0; i<m; i++) {
    scan("%d", &avail[i]);
}

```

```

int work[m];

```

```

for(i=0; i<m; i++) {
    work[i] = avail[i];
}

```

```

for(i=0; i<n; i++) {
    int flag = 0;

```

```

    for(j=0; j<m; j++) {
        if(alloca[i][j] != 0) {
            flag = 1;
            break;
        }
    }
}

```



```

if (flag == 0)
    finish[i] = 1;
else
    finish[i] = 0;
}

```

```

int count = 0;
while (count < n) {
    int found = 0;
    for (i = 0; i < n; i++) {
        if (finish[i] == 0) {
            int possible = 1;
            for (j = 0; j < m; j++) {
                if (request[i][j] > work[j]) {
                    possible = 0;
                    break;
                }
            }
            if (possible) {
                for (k = 0; k < m; k++) {
                    work[k] += alloc[i][k];
                }
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
}

```



```
if (found == 0)
    break;
```

```
int deadlock = 0;
```

```
for (i = 0; i < n; i++) {
```

```
    if (finish[i] == 0) {
        deadlock = 1;
```

```
        break;
```

```
    }
}
```

```
if (deadlock == 0) {
```

```
    printf("Safe seq is in safe state");
```

```
    printf("Safe Sequence is: ");
```

```
for (i = 0; i < count; i++) {
```

```
    printf("P%d", safe_seq[i]);
```

```
    if (i == count - 1)
```

```
        printf("\n->");
```

```
}
```

```
else {
```

```
    printf("\n In Deadlock detected");
```

```
    printf("Deadlock process are:");
```

```
    for (i = 0; i < n; i++) {
```

```
        if (finish[i] == 0)
```

```
            printf("P%d", i);
```

```
    }
```

```
    return 0;
```

```
}
```


SAMPLE OUTPUT :

Enter the number of Process: 5

Enter the number of resource: 3

Enter allocation matrix:

Process 0: 0 1 0

Process 1: 2 0 0

Process 2: 3 0 3

Process 3: 2 1 1

Process 4: 0 0 2

Enter the request matrix:

Process 0: 0 0 0

Process 1: 2 0 2

Process 2: 0 0 0

Process 3: 1 0 0

Process 4: 0 0 2

Enter the available resource: 0 0 0

System is in Safe State

Safe sequence is $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

System is in Safe State

Safe sequence is $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

System is in Safe State

Safe sequence is $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

System is in Safe State

Safe sequence is $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

System is in Safe State

Safe sequence is $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

System is in Safe State

Safe sequence is $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

System is in Safe State

Safe sequence is $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

System is in Safe State

Safe sequence is $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

System is in Safe State

Safe sequence is $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

System is in Safe State

Program - 6

Question:

Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-Fit b) Best-Fit c) First-Fit

CODE

```
#include <stdio.h>

void copyArray(int source[], int dest[], int n){
    for(int i=0;i<n;i++){
        dest[i]=source[i];
    }
}

void firstFit(int blocks[], int m, int process[], int n){
    int allocation[n];

    for(int i=0;i<n;i++){
        allocation[i]=-1;
    }

    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            if(blocks[j]>=process[i]){
                allocation[i]=j;
                blocks[j]-=process[i];
                break;
            }
        }
    }
}
```



```

printf("\n--- First Fit ---\n");
printf("Process No\tProcess Size\tBlock No\n");

for(int i=0;i<n;i++){
    printf("%d\t%d\t",i+1,process[i]);

    if(allocation[i]!=-1){
        printf("%d\n",allocation[i]+1);
    }
    else{
        printf("Not Allocated\n");
    }
}
}

void bestFit(int blocks[], int m, int process[], int n){
    int allocation[n];

    for(int i=0;i<n;i++){
        allocation[i]=-1;
    }

    for(int i=0;i<n;i++){
        int bestIdx=-1;

        for(int j=0;j<m;j++){
            if(blocks[j]>=process[i]){
                if(bestIdx==-1 || blocks[j]<blocks[bestIdx]){
                    bestIdx=j;
                }
            }
        }
    }
}

```

```

        }
    }
}

if(bestIdx!=-1){
    allocation[i]=bestIdx;
    blocks[bestIdx]-=process[i];
}
}

printf("\n--- Best Fit ---\n");
printf("Process No\tProcess Size\tBlock No\n");

for(int i=0;i<n;i++){
    printf("%d\t%d\t",i+1,process[i]);

    if(allocation[i]!=-1){
        printf("%d\n",allocation[i]+1);
    }
    else{
        printf("Not Allocated\n");
    }
}
}

void worstFit(int blocks[], int m, int process[], int n){
    int allocation[n];

    for(int i=0;i<n;i++){

```



```

        allocation[i]=-1;
    }

    for(int i=0;i<n;i++){
        int worstIdx=-1;

        for(int j=0;j<m;j++){
            if(blocks[j]>=process[i]){
                if(worstIdx==-1 || blocks[j]>blocks[worstIdx]){
                    worstIdx=j;
                }
            }
        }

        if(worstIdx!=-1){
            allocation[i]=worstIdx;
            blocks[worstIdx]-=process[i];
        }
    }

    printf("\n--- Worst Fit ---\n");
    printf("Process No\tProcess Size\tBlock No\n");
    for(int i=0;i<n;i++){
        printf("%d\t%d\t",i+1,process[i]);

        if(allocation[i]!=-1){
            printf("%d\n",allocation[i]+1);
        }
        else{
            printf("Not Allocated\n");
        }
    }
}

```

```
}
```

```
int main(){  
    int m,n;  
    printf("Enter number of memory blocks: ");  
    scanf("%d",&m);  
    int pBlk[m], bBlk[m];  
    printf("Enter sizes of memory blocks:\n");  
    for(int i=0;i<m;i++){  
        scanf("%d",&pBlk[i]);  
    }  
    printf("Enter number of processes: ");  
    scanf("%d",&n);  
    int proc[n];  
  
    printf("Enter sizes of processes:\n");  
  
    for(int i=0;i<n;i++){  
        scanf("%d",&proc[i]);  
    }  
    copyArray(pBlk,bBlk,m);  
    firstFit(bBlk,m,proc,n);  
    copyArray(pBlk,bBlk,m);  
    bestFit(bBlk,m,proc,n);  
    copyArray(pBlk,bBlk,m);  
    worstFit(bBlk,m,proc,n);  
  
    return 0;  
}
```


Result

```
1WA24CS223
Priyanshu Sharan
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

--- First Fit ---
Process No      Process Size      Block No
1                212                2
2                417                5
3                112                2
4                426              Not Allocated

--- Best Fit ---
Process No      Process Size      Block No
1                212                4
2                417                2
3                112                3
4                426                5

--- Worst Fit ---
Process No      Process Size      Block No
1                212                5
2                417                2
3                112                5
4                426              Not Allocated
priyanshusharan@Priyanshus-MacBook-Air os-coding %
```

Lab Program 6 : Write a program to simulate

i) First Fit

ii) Best Fit

iii) Worst Fit

First Fit → Allocate the first hole which is big enough

→ Searching starts from beginning

Best Fit → Allocate the smallest hole that is big enough

→ Search the entire list

Worst Fit → Allocate the largest hole

Q Given 5 free partition of 100KB, 500KB, 200KB, 300KB, 600KB using First, Best and worst Fit Alg to place 212KB, 417KB, 112KB, 426KB

600KB	417	600	426	600	212
300KB		300	212	300	112
200KB	112	200	112	200	
500KB	212	500	47	500	417
100KB		100		100	

First
426 cannot
be allocated

Best

Worst
426 cannot
be allocated

Q. Given 8 free partition of 400K, 700K, 200K, 300K, 600K place the process
212KB 512KB 312KB, 526KB

600	312	600	512	600	512
300		300	212	300	
200		200		200	
700	512	700	526	700	212
400	212	400	312	400	312

First Fit Best Fit Worst Fit

526 cannot be allocated All are allocated 526 cannot be allocated

//C-CODE

#include <stdio.h>

```
void copyArray (int source[], int dest[],
                int n) {
    for (int i=0; i<n; i++) {
        dest[i] = source[i];
    }
}
```

```
void firstFit (int blocks[], int m,
               int processes[], int n) {
    int allocation[n];
    for (int i=0; i<n; i++) {
        allocation[i] = -1;
    }
}
```



```

for (int i=0; i<n; i++) {
    for (int j=0; j<m; j++) {
        if (blocks[j] >= process[i]) {
            allocation[i] = j;
            blocks[j] -= process[i];
            break;
        }
    }
}

printf("\n --- First Fit --- \n");
printf("%-11s %-15s %-2s\n", "Process No.", "Process Size", "Block No.");
for (int i=0; i<n; i++) {
    printf("%-11d %-15d", i+1, process[i]);
    if (allocation[i] != -1) {
        printf("%-2d\n", allocation[i]);
    } else {
        printf("Not Allocated\n");
    }
}

```

```

void bestFit(int blocks[], int m,
int processes[], int n) {
    int allocation[n];
    for (int i=0; i<n; i++) {
        allocation[i] = -1;
    }
    for (int i=0; i<n; i++) {
        int bestIdx = -1;
        for (int j=0; j<m; j++) {

```



```

        if (block[j] >= process[i]) {
            if (bestIdx == -1 || block[j] <
                block[bestIdx]) {
                bestIdx = j;
            }
        }
    }

```

```

    if (bestIdx != -1) {
        allocation[i] = bestIdx;
        block[bestIdx] -= process[i];
    }

```

```

    printf("\n --- Best Fit --- \n");
    printf("%-11s %-16s %-8s\n", "Process No",
        "Process Size", "Block No");

```

```

    for (int i = 0; i < n; i++) {
        printf("%-11d %-16d", i+1, process[i]);
        if (allocation[i] != -1) {
            printf("%-8d\n", allocation[i]+1);
        } else {
            printf("\n Not Allocated\n");
        }
    }

```

```

void worstFit (int blocks[], int m, int
    process[], int n) { int allocation[n];
    for (int i = 0; i < n; i++) { allocation[i] = -1;
    }
    for (int i = 0; i < n; i++) { int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (process[i] <= blocks[j] &&
                (worstIdx == -1 || blocks[j] >
                    blocks[worstIdx])) {
                worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blocks[worstIdx] -= process[i];
        }
    }
}

```



```

        if (worstFit == -1) {
            allocation[i] = -1;
            blocks[worstFit] = process[i];
        }
    }
    printf("In --- Worst fit ---\n");
    printf("%s Process No. Process Size Block No.\n");
    for (int i = 0; i < n; i++) {
        if (allocation[i] != -1)
            printf("%s %d %d %d\n", i+1, process[i],
                allocation[i]+1);
        else printf("%s %d %d Not Allocated, %d\n",
            i+1, process[i], i+1);
    }
}

int main() {
    int m, n;
    printf("Enter no. of memory block, their sizes\n");
    scanf("%d", &m);
    int pBlock[m], cBlock[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &pBlock[i]);
    }
    printf("Enter no. of processes their sizes\n");
    scanf("%d", &n);
    int proc[n];
    for (int i = 0; i < n; i++) scanf("%d", &proc[i]);
    copyArray(pBlock, cBlock, m);
    firstFit(cBlock, m, proc, n);
    copyArray(pBlock, cBlock, m);
    bestFit(cBlock, m, proc, n);
    copyArray(pBlock, cBlock, m);
    worstFit(cBlock, m, proc, n);
    return 0;
}

```


Output

Enter number of memory blocks: 5

Enter sizes of 5 memory blocks

100 500 200 300 600

Enter number of processes: 4

Enter sizes of 4 processes

--- First Fit ---

Process No	Process Size	Block No
1	212	1
2	417	5
3	112	3
4	426	Not Allocated

--- Best Fit ---

Process No	Process Size	Block No
1	212	4
2	417	2
3	112	3
4	426	5

--- Worst Fit ---

Process No	Process Size	Block No
1	212	5
2	417	2
3	112	4
4	426	Not Allocated

14/5/21

Program - 7

Question:

Write a C program to simulate page replacement algorithms. (Any one)

a) FIFO b) LRU c) Optimal

CODE

```
#include <stdio.h>
#include <stdlib.h>

void printFrames(int frames[], int size){
    printf("[");

    for(int i=0;i<size;i++){
        printf("%d",frames[i]);

        if(i<size-1){
            printf(" ");
        }
    }

    printf("\n");
}

void fifo(int pages[], int num_pages, int num_frames){
    int *frames=(int *)malloc(num_frames*sizeof(int));

    int frame_count=0;
    int next=0;
    int page_faults=0;

    printf("\n--- FIFO Page Replacement ---\n");

    for(int i=0;i<num_pages;i++){
        int page=pages[i];
```



```

    int found=0;

    for(int j=0;j<frame_count;j++){
        if(frames[j]==page){
            found=1;
            break;
        }
    }

    if(!found){
        if(frame_count<num_frames){
            frames[frame_count]=page;
            frame_count++;
        }
        else{
            frames[next]=page;
            next=(next+1)%num_frames;
        }

        page_faults++;
    }

    printf("Page %d -> ",page);
    printFrames(frames,frame_count);
}

printf("\nTotal Page Faults (FIFO): %d\n",page_faults);

free(frames);
}

void lru(int pages[], int num_pages, int num_frames){
    int *frames=(int *)malloc(num_frames*sizeof(int));

```

```

int frame_count=0;
int page_faults=0;

printf("\n--- LRU Page Replacement ---\n");

for(int i=0;i<num_pages;i++){
    int page=pages[i];
    int found_idx=-1;

    for(int j=0;j<frame_count;j++){
        if(frames[j]==page){
            found_idx=j;
            break;
        }
    }

    if(found_idx!=-1){
        int temp=frames[found_idx];

        for(int j=found_idx;j<frame_count-1;j++){
            frames[j]=frames[j+1];
        }

        frames[frame_count-1]=temp;
    }
    else{
        if(frame_count<num_frames){
            frames[frame_count]=page;
            frame_count++;
        }
        else{
            for(int j=0;j<num_frames-1;j++){

```



```

        frames[j]=frames[j+1];
    }

    frames[num_frames-1]=page;
}

    page_faults++;
}

    printf("Page %d -> ",page);
    printFrames(frames,frame_count);
}

printf("\nTotal Page Faults (LRU): %d\n",page_faults);

free(frames);
}

void optimal(int pages[], int num_pages, int num_frames){
    int *frames=(int *)malloc(num_frames*sizeof(int));

    int frame_count=0;
    int page_faults=0;

    printf("\n--- Optimal Page Replacement ---\n");

    for(int i=0;i<num_pages;i++){
        int page=pages[i];
        int found=0;

        for(int j=0;j<frame_count;j++){
            if(frames[j]==page){
                found=1;

```

```

        break;
    }
}

if(!found){
    if(frame_count<num_frames){
        frames[frame_count]=page;
        frame_count++;
    }
    else{
        int victim_idx=-1;
        int farthest=-1;

        for(int j=0;j<num_frames;j++){
            int next_use=-1;
            for(int k=i+1;k<num_pages;k++){
                if(pages[k]==frames[j]){
                    next_use=k;
                    break;
                }
            }

            if(next_use==-1){
                victim_idx=j;
                break;
            }

            if(next_use>farthest){
                farthest=next_use;
                victim_idx=j;
            }
        }
        frames[victim_idx]=page;
    }
}

```



```

        }
        page_faults++;
    }
    printf("Page %d -> ",page);
    printFrames(frames,frame_count);
}
printf("\nTotal Page Faults (Optimal): %d\n",page_faults);
free(frames);
}

```

```

int main(){
    printf("1WA24CS223\n");
    printf("Priyanshu Sharan\n");
    int num_pages;
    int num_frames;
    printf("Enter number of pages: ");
    scanf("%d",&num_pages);
    int *pages=(int *)malloc(num_pages*sizeof(int));
    printf("Enter page reference string:\n");
    for(int i=0;i<num_pages;i++){
        scanf("%d",&pages[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d",&num_frames);

    fifo(pages,num_pages,num_frames);
    lru(pages,num_pages,num_frames);

    optimal(pages,num_pages,num_frames);
    free(pages);

    return 0;
}

```

Result

```
1WA24CS223
Priyanshu Sharan
Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]

Total Page Faults (FIFO): 10

--- LRU Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [0 1 2]
Page 0 -> [1 2 0]
Page 3 -> [2 0 3]
Page 0 -> [2 3 0]
Page 4 -> [3 0 4]
Page 2 -> [0 4 2]
Page 3 -> [4 2 3]
Page 0 -> [2 3 0]
Page 3 -> [2 0 3]

Total Page Faults (LRU): 9

--- Optimal Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [2 4 3]
Page 2 -> [2 4 3]
Page 3 -> [2 4 3]
Page 0 -> [0 4 3]
Page 3 -> [0 4 3]

Total Page Faults (Optimal): 7
priyanshusharan@Priyanshus-MacBook-Air os-coding %
```


LAB PROGRAM - 7

Write lab program to simulate
page replacement algorithms:

1) FIFO

2) LRU

3) Optimal

1) FIFO :- First in First Out

strings		7	0	1	2	0	3	0	4	2	3	0	3	1	2	0
main	f1			1	1	1	1	0	0	0	3	3	3	3	2	2
memory	f2		0	0	0	0	3	3	3	2	2	2	2	1	1	1
frames	f1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0
		*	*	*	*	hit	*	*	*	*	*	*	*	hit	*	hit

no of frames = 5

hit = 3

(*) miss = 12

Page fault

$\Rightarrow$ no of hit / No of pages = 3/15

$\Rightarrow$ no of miss / No of pages = 12/15

2) Optimal Page Replacement

	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
f4				2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
f3			1	1	1	1	1	4	4	4	4	4	4	4	4	4	1	1	1	1
f2		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
f1	7	7	7	7	7	3	3	3	3	3	3	3	3	7	7	7	7	7	7	7
	*	*	*	*	hit	*	hit	*	hit	hit	hit	hit	hit	hit	*	hit	hit	*	hit	hit

Replace the page which is not used in longest time in future

Hit Ratio $\Rightarrow \frac{\text{No. of hit}}{\text{Total no page}} = 12/20$

Miss Ratio $\Rightarrow \frac{\text{no of miss}}{\text{Total no of page}} = 8/20$

Hit = 12

Miss = 8

Least Recently Used (LRU)

0	5	1	0	0	8	8	1	0	8	0	0	1	0	8	0	1	0	8	0
5	7	0	1	2	0	8	0	4	2	8	0	8	2	1	2	0	1	7	0
4	1	1	1	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
8	0	0	1	1	1	1	1	4	4	4	4	4	4	1	1	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	7	7	7	7	7	8	8	8	8	8	8	8	8	8	8	8	8	7	7
	*	*	*	*	H	*	H	*	H	H	H	H	H	*	H	H	H	*	H

Replace the least recently used page in past

hit = 12 miss = 8

Hit ratio = $\frac{\text{no of hit}}{\text{pages}} = \frac{12}{20} = \frac{3}{5}$

Miss ratio = $\frac{\text{no of miss}}{\text{no of pages}} = \frac{8}{20} = \frac{2}{5}$

Q) Given returns to the following pages
5, 0, 1, 0, 2, 3, 0, 2, 4, 3, 3, 2, 0, 2, 1, 2, 1, 0, 1, 0

Find Hit ratio & Miss ratio for

a) AFO

b) Optimal

c) LRU

a) AFO

~~5 0 1 0 2 3 0 2 4 3 3 2 0 2 1 2 7 0 1 1 0~~

	5	0	1	0	2	3	0	2	4	3	3	2	0	2	1	2	7	0	1	1	0
f3			1	1	1	1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	
f2		0	0	0	0	3	3	3	3	3	3	2	2	2	2	2	2	0	0	0	0
f1	5	5	5	5	2	2	2	2	4	4	4	4	4	4	4	4	7	7	7	7	7
	*	*	*	H	*	*	*	H	*	H	H	*	H	H	*	H	*	*	H	H	H

$$\text{Hit Ratio} = \frac{\text{no of hit}}{\text{no of page}} = \frac{10}{21}$$

$$\text{Miss ratio} = \frac{\text{no of miss}}{\text{no of page}} = \frac{11}{21}$$

b) Optimal Page Replacement

	5	0	1	0	2	3	0	2	4	3	3	2	0	2	1	2	7	0	1	1	0
f3			1	1	1	3	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
f2		0	0	0	0	0	0	0	4	4	4	4	0	0	0	0	0	0	0	0	0
f1	5	5	5	5	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7	7	7
	*	*	*	H	*	*	H	H	*	H	H	H	*	H	*	H	*	H	*	H	H

$$\text{Hit ratio} = \frac{12}{21}$$

$$\text{Miss Ratio} = \frac{9}{21}$$

Q) LRU.

	5	0	1	0	2	3	0	2	4	3	3	2	0	2	1	2	7	0	1	1	1
J3			1	1	1	3	3	3	4	4	4	4	0	0	0	0	7	7	7	7	2
J2		0	0	0	0	0	0	0	0	3	3	3	3	3	1	1	1	0	0	0	0
J1	5	5	5	5	2	2	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1
	x	x	x	x	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*

$$\text{Hit ratio} = \frac{\text{no of hit}}{\text{no of pages}} = \frac{9}{21}$$

$$\text{Miss ratio} = \frac{\text{no of miss}}{\text{ratio}} = \frac{12}{21}$$

C-CODE

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void print_frames(int frames[], int size) {
```

```
    printf("[");
```

```
    for (int i=0; i<size; i++) {
```

```
        printf("%d",
```

```
        frames[i]);
```

```
        if (i<size-1) {
```

```
            printf(" ");
```

```
        }
```

```
    printf("] \n");
```

```
}
```



```

void fifo (int pages[], int num-pages,
           int num-frames) {
    int *frames = (int *) malloc (num-frames
    * sizeof(int));
    int frame-count = 0;
    int next = 0;
    int page-faults = 0;

```

```

    printf("\n --- FIFO Page Replacement --- \n");

```

```

    for (int i=0; i<num-pages; i++) {

```

```

        int page = pages[i];
        int found = 0;

```

```

        for (int j=0; j<frame-count; j++) {
            if (frames[j] == page) {

```

```

                found = 1;
                break;

```

```

            }
        }

```

```

        if (!found) {

```

```

            if (frame-count < num-frames) {

```

```

                frames[frame-count] = page;
                frame-count++;

```

```

            } else {

```

```

                frames[next] = page;

```

```

                next = (next + 1) % num-frames;

```

```

            }

```

```

            page-faults++;

```

```

        }

```

```

    }

```



```

    printf("Page %d -> ", page);
    print_frames(frames, frame_count);
}
printf("Total page faults (FIFO): %d\n",
       page_faults);
free(frames);

```

```

void lru(int pages[], int num_pages,
         int num_frames) {
    int *frames = (int *) malloc (num_frames
                                   * sizeof(int));
    int frame_count = 0;
    int page_fault = 0;

    printf("\n--- LRU Page Replacement ---\n");

    for(int i = 0; i < num_pages; i++) {
        int page = pages[i];
        int found_idx = -1;

        for(int j = 0; j < frame_count; j++) {
            if(frames[j] == page) {
                found_idx = j;
                break;
            }
        }

        if(found_idx != -1) {
            int tmp = frames[found_idx];
            for(int j = found_idx; j < frame_count - 1; j++) {
                frames[j] = frames[j+1];
            }
            frames[frame_count - 1] = tmp;
        }
        frame_count++;
        page_fault++;
    }

    printf("Total page faults (LRU): %d\n", page_fault);
}

```



```

frames[j] = frames[j+1];
}
frames[frame-count-1] = page;
} else {
    if (frame-count < num-frames) {
        frames[frame-count] = page;
        frame-count++;
    } else {
        for (int j = 0; j < num-frames-1; j++) {
            frames[num-frames-1] = page;
        }
        page-faults++;
    }
}
printf("Page %d -> ", page);
printf-frames(frames, frame-count);
printf("Total Page Faults (LRU): %d\n", page-faults);
tree(frames);
}

void optimal(int page[], int num-pages,
            int num-frames) {
    int *frames = (int *) malloc (num-frames *
                                   sizeof(int));
    int frame-count = 0;
    int page-faults = 0;

```


print("Optimal Page Replacement
--- in ---")

```
for (int i = 0; i < num_pages; i++) {
```

```
    int page = pages[i];
```

```
    int found = 0;
```

```
    for (int j = 0; j < frame_count; j++) {
```

```
        if (frames[j] == page) {
```

```
            found = 1;
```

```
            break;
```

```
        if (!found) {
```

```
            if (frame_count < num_frames) {
```

```
                frames[frame_count++] = page;
```

```
                frame_count++;
```

```
            } else {
```

```
                int victim_idx = -1;
```

```
                int lastuse = -1;
```

```
                for (int j = 0; j < num_frames; j++) {
```

```
                    int lastuse = -1;
```

```
                    for (int k = i+1; k < num_pages;
```

```
                        k++) {
```

```
                            if (pages[k] == frames[j]) {
```

```
                                lastuse = k;
```

```
                            }  
                        }  
                    }  
                    if (lastuse == -1) {
```

```
                        victim_idx = j;
```

```
                    }  
                }  
                frames[victim_idx] = page;
```



```

        if (next_use == -1) {
            victim_idx = j;
            break;
        }
        if (next_use > last_use) {
            last_use = next_use;
            victim_idx = j;
        }
        frames[victim_idx] = page;
    }
    page_faults++;
}
printf("Page %d -> ", page);
print_frames(frames, frame_count);
}
printf("Total Page Faults (optional): %d\n",
       page_faults);
tree(frames);

```

```

int main() {
    int num_pages, num_frames;
    printf("Enter number of pages: ");
    if (scanf("%d", &num_pages) != 1) return 1;

    int *pages = (int *) malloc(num_pages *
                                sizeof(int));
    printf("Enter page replacement string: \n");
    for (int i = 0; i < num_pages; i++) {
        if (scanf("%d", &pages[i]) != 1) return 1;
    }
}

```



```
printf("Enter number of frames: ");
if (scanf("%d", &num_frames) != 1)
    return 1;
```

```
fifo(pages, num_pages, num_frames);
lru(pages, num_pages, num_frames);
optimal(pages, num_pages, num_frames);
```

```
free(pages);
```

```
return 0;
```

SAMPLE OUTPUT

Enter number of pages: 12

Enter page reference string:

7 0 1 2 0 3 0 4 2 3 0 3

Enter number of frames: 3

--- FIFO Page Replacement ---

Page 7 → [7]

Page 0 → [7 0]

Page 1 → [7 0 1]

Page 2 → [2 0 1]

Page 0 → [2 0 1]

Page 3 → [2 3 1]

Page 0 → [2 3 0]

Page 4 → [4 3 0]

Page 2 → [4 2 0]

Page 3 → [4 2 3]

Page 0 → [0 2 3]

Page 3 → [0 2 3]

Total Page Faults (FIFO): 10

--- LRU Page Replacement ---

Page 7 → [7]

Page 0 → [7 0]

Page 1 → [7 0 1]

Page 2 → [0 1 2]

Page 0 → [1 2 0]

Page 3 → [2 0 3]

Page 0 → [2 3 0]

1) Page 4 → [3 0 4]

2) Page 2 → [0 4 2]

Page 3 → [4 2 3]

Page 0 → [2 3 0]

Page 3 → [2 0 3]

Total Page Fault (LRU): 9

--- OPTimal Page Replacement ---

Page 7 → [7]

Page 0 → [7 0]

Page 1 → [7 0 1]

Page 2 → [2 0 1]

Page 0 → [2 0 1]

Page 3 → [2 0 3]

Page 0 → [2 0 3]

Page 4 → [2 4 3]

Page 2 → [2 4 3]

Page 3 → [2 4 3]

Page 0 → [0 4 3]

Page 3 → [0 4 3]

Total Page Faults (Optimal): 7